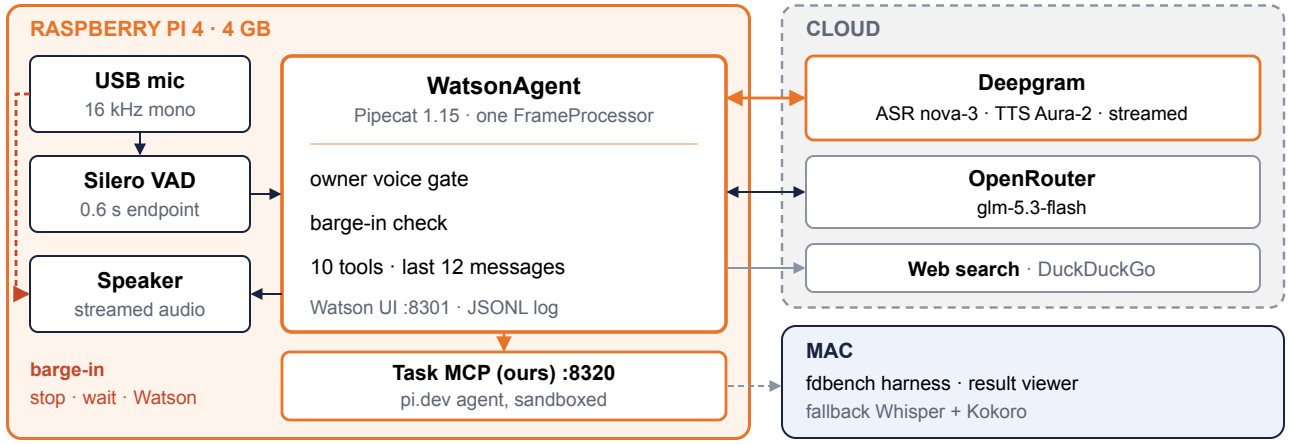
Full-Duplex Voice Agent on a Raspberry Pi

sections 1–6 in priority order

1 SYSTEM DIAGRAM *the one diagram that explains the build*




```
mic → VAD (RPi) → ASR (Deepgram) → agent + tools (RPi) → LLM (cloud) → TTS (Deepgram) → speaker
```

barge-in: owner voice + stop word → flush TTS + drop LLM stream → listen

2 PROMPT *in the repo, versioned: brain.py v4 (Nov 6, owner-approved)*

SYSTEM PROMPT (brain.py v4) · full text, verbatim

You are Watson, a voice assistant in a live spoken conversation. Reply in one or two short spoken sentences, plain text, no markdown or emoji; give more detail only when asked.

When a request is clear, act with the right tool without offering first. If a needed detail is missing, ask one short question.

Handle music, lights, thermostat, timers, alarms, reminders and messages yourself with their tools.

Say an action is done only after the tool confirms it; say clearly if it is pending or failed.

For news, facts you are unsure of, and the time in other cities, use web_search; never guess. If the search doesn't settle it, say so briefly.

Never mention tools or internal details.

After an interruption, follow the user's latest request; an action already submitted still stands unless they cancel it.

Big jobs (sites, apps, research, files) go to the background agent: reply DELEGATE: <one-line request>. Never speak the tag. If the job has no topic yet ("build me a website"), ask what it should be about first.

Owner-approved v4 (lines 1–7). The last line (teal) is the runtime DELEGATE protocol the voice loop parses. Barge-in is enforced in code; a filter strips paths, file names and URLs from everything Watson says. Bench mode appends an addressed-speech rule and a 35-word cap.

- Mic → VAD on the RPi → **Deepgram ASR** (nova-3, live websocket fed while the user talks) → agent on the RPi → **Deepgram TTS** (Aura-2 Jupiter, audio played as it streams in) → speaker on the RPi.
- Harness: **Pipecat 1.15**, one FrameProcessor (WatsonAgent). Stock transports only; the same agent runs the live mic and the benchmark socket.
- LLM: **z-ai/glm-5.3-flash** via OpenRouter, streaming with function calling. 20 s read timeout; an empty stream goes straight to **gpt-4o-mini**. Tool turns say "One sec." while the tool runs.
- **Task MCP on the RPi:** tasks_mcp.py (streamable HTTP) takes big jobs and web lookups. Jobs run the **pi.dev** agent under bubblewrap, writing only in ~/sandbox; the Watson UI (ui_server.py, also on the Pi) serves the result and the Mac opens it; Watson announces it at the next pause, never reading a path.
- **Deviation from the brief:** ASR and TTS are commercial APIs (Deepgram) called directly from the RPi, not through our own MCP servers. Quick tools (lights, timers) are in-process calls. The Mac's Whisper/Kokoro server stays as a fallback.
- Second MCP built by us: **tts_mcp.py** (EfficientSpeech Tiny, tools `speak`, `health`). Tested on the Mac; not in the live loop.
- Barge-in cancels playback (InterruptionFrame) and the LLM stream (generation counter). The unplayed audio is stashed so a "yeah" backchannel can resume it.
- Boundary: RPi = audio I/O, VAD, gates, agent loop, tools, task MCP + pi.dev agent, Watson UI. Mac = benchmark harness, result viewer, fallback ASR/TTS. Cloud = Deepgram, LLM, search engine.

3 CONTEXT ENGINEERING

GOES IN:

- System prompt v4 (~230 tokens) + 10 tool schemas (~780 tokens)
- Final Deepgram transcript per turn, wake word stripped (no partials)
- Tool results as tool messages, for that turn only
- Conversation history (rolling)
- VAD turn events drive the loop; they stay out of the context
- Memory: in-conversation only, no RAG

MANAGED:

- History capped at the last 12 messages, cut at a user turn so no tool reply loses its call
- Why: the first token arrives later as the context grows
- Tool calls and results dropped once the turn is answered
- idle (5 min) → listen → think → speak
- On barge-in: generation++, stale audio and stream dropped
- Measured (fdbench): 1,665 prompt tokens per call median (max 2,107), 12 messages (max 15)

MCP Services, Evaluation & Model Serving

what runs where, measured on the RPi

4 MCP SERVICES *+ section 6: model serving (one table, one truth)*

Component	Server / transport	Tools	Model	Served where / notes
ASR	Deepgram `/v1/listen`, HTTPS **(not MCP)**	listen(wav) → text	nova-3; keyterms "Watson" + demo words	called from the RPi; live websocket fed during speech, transcript ~0.25 s after the endpoint
TTS	Deepgram `/v1/speak`, HTTPS; MCP: tts_mcp.py (stdio, HTTP)	speak(text) → wav; MCP: speak, health	Aura-2 aura-2-jupiter-en; MCP: EfficientSpeech Tiny	called from the RPi; streamed per sentence, played as it arrives; flushed on barge-in
LLM	WatsonAgent (Pipecat harness), OpenAI-compatible streaming	function calling, 10 tools	z-ai/glm-5.3-flash; fallback openai/gpt-4o-mini	Cloud (OpenRouter); 20 s read timeout, empty stream → fallback
VAD	in-process	SileroVADAnalyzer(frames)	Silero VAD (ONNX)	RPi on-device, 16 kHz mono; start 0.1 s, stop 0.6 s
Speaker ID	in-process (speakerid.py)	similarity(pcm, profile)	resemblyzer (GE2E)	RPi; answers the session holder (≥ 0.60); saying "Watson" claims the session; echo check vs Watson's voice (0.44)
Tasks	**tasks_mcp.py** / streamable HTTP :8320 (MCP)	submit_task(request), list_tasks(), finished(), browse(query)	pi.dev 1.5 agent on glm-5.3-flash; search: DuckDuckGo (ddgs)	RPi; agent under bubblewrap in ~/sandbox; result opens on the Mac via the Watson UI; 10 s for a one-page site
Quick tools	in-process `brain.execute_tool` **(not MCP)**	get_time, web_search, music, lights, thermostat, timers, alarms, reminders	n/a	RPi; home state is virtual (JSON); web_search goes through browse (spoken only)

5 EVALUATION & BENCHMARK *standardized latency method*

- One clock: the Pi's `time.monotonic`; Watson sends a stage mark at each step over the benchmark socket.
- Stages: `vad_end → asr_final → llm_first_token → tts_first_audio → audio_out`. E2E is measured by fdbench from the audio itself.
- 30 timed turns (10 phrases × 3) + Full-Duplex-Bench, 3 rounds (30 trials). Assistant on the Pi, harness on the Mac over Wi-Fi.
- Full-Duplex-Bench (n = 3 per group): interruptions handled 3/3 (judge 5.0/5), backchannel resume 2/3, pause take-overs 1/6, turn-taking missed 1/3, answered speech not meant for it 2/6 (side talk; TV, noise and background 0).

Metric	Definition	Target	Mean	p95	Median
E2E latency	end of speech → audio out	4.0 s	3.20	6.27	2.57
ASR final	end of speech → transcript	0.8 s	0.34	0.42	0.28
LLM TTFT	transcript → first token	1.5 s	2.48	4.78	2.02
TTS TTFB	text → first audio	0.6 s	0.43	1.45	0.30
Barge-in	overlap → playback stopped	0.8 s	0.77 / 0.79 s (mean / p95, 3 trials)		
WER	ASR accuracy, 10 phrases × 3	0.15	0.11	30 turns	

Seconds, fdbench run 2026-11-07_11-22-33 on the Pi. Targets: course defaults. The E2E p95 comes from language-model spikes (LLM p95 4.78 s).

DEMO (run sheet: docs/demo-spiel.html)

1. "Watson" → "Yes? I'm listening." (also "button", "Hudson")
2. "Build me a website about Manila street food" → task MCP, pi.dev
3. Meanwhile: "Who won the 2026 NBA finals?" → spoken answer
4. Meanwhile: "Lights off" → lights_control
5. "Done! The ... website is ready" → page opens by itself
6. "Stop" → barge-in; classmate says "Watson, …" → takes over

No push-to-talk. Slow API: 20 s timeout; an empty reply goes to gpt-4o-mini.

RESULTS

Stage medians on the Pi (E2E median 2.57 s; target 4.0 s):



Pi: Raspberry Pi 4 Model B Rev 1.1, 4 GB, no fan, peak 74 °C, never throttled; Wi-Fi to the harness. **Date:** Nov 7, 2026. **Models:** Deepgram nova-3 + aura-2-jupiter-en (both streamed), glm-5.3-flash, Silero VAD, Pipecat 1.15.

CHALLENGES & REFERENCES

1. **Echo as loud as the owner**, AEC gave 0–4 dB → decide by speaker identity; Watson's voice scores 0.44 vs the 0.60 gate; stranger playback rejected (0.45, 0.47).
2. **Accent: "Watson" heard as "button", "Hudson"** → high-recall wake matcher; caught wakes in the logs 81 → 87, no new false wakes; 74/74 tests.
3. **Profile overwrite locked the owner out** (0.52–0.58) → refresh blends 10%; profile stays at 0.994 similarity after a hostile refresh.

modelcontextprotocol.io · docs.pipecat.ai · github.com/airimonda/me3-full-duplex